

Data Stage and Scoping

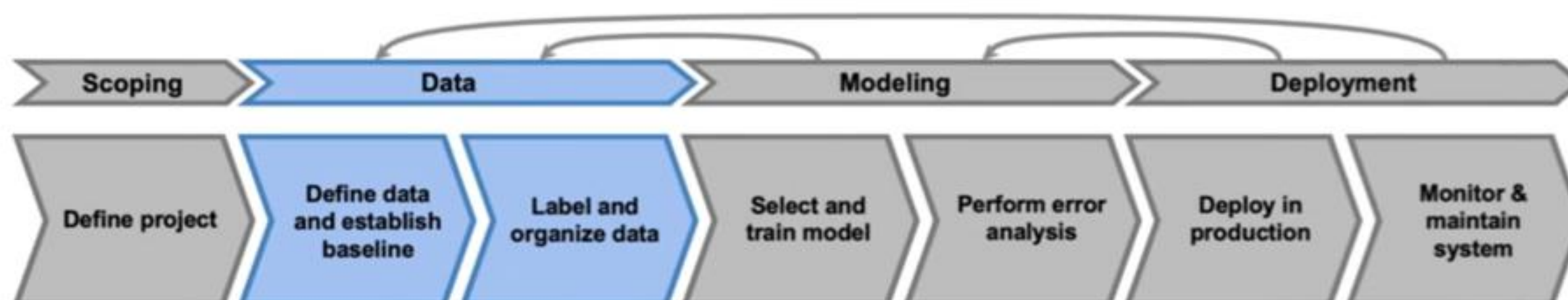
From Machine Learning Models to Production Systems

Nguyen Manh Toan

Swinburne Vietnam

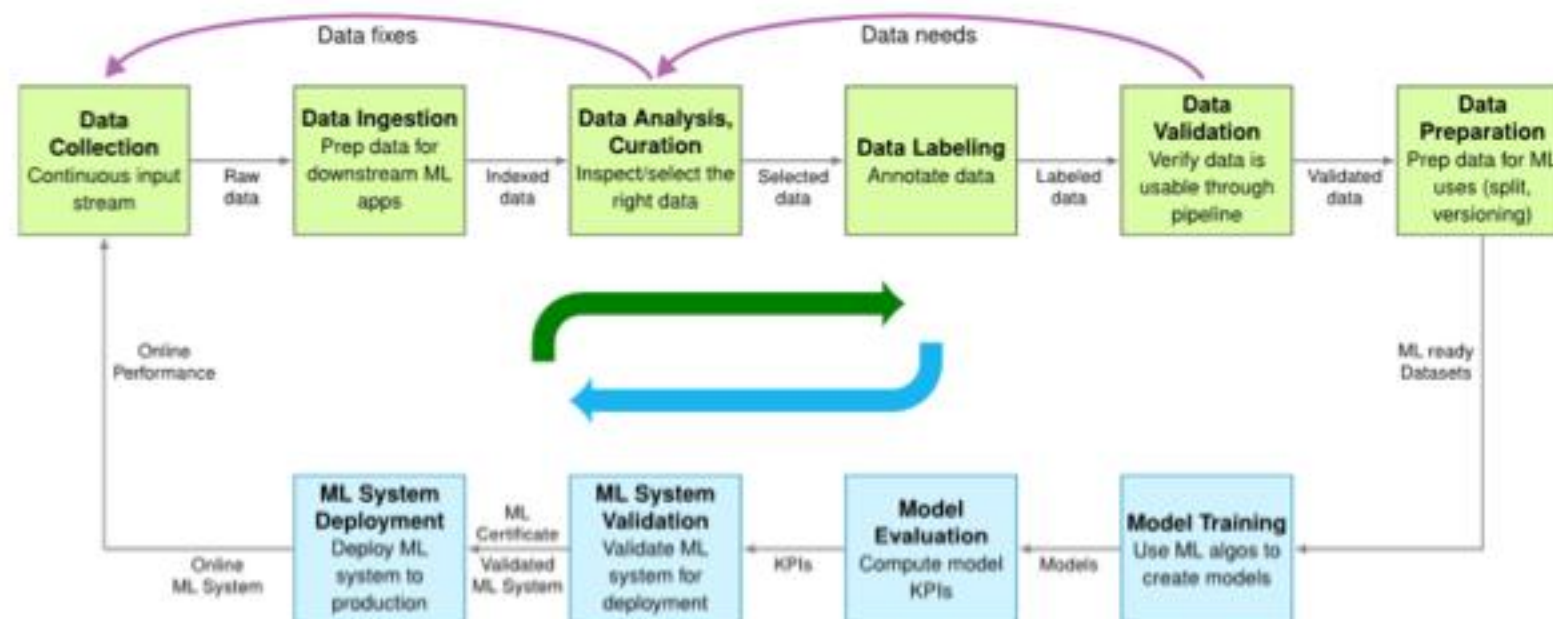
January, 2026

Data Stage



- Second stage in the ML lifecycle, where raw data is gathered, processed, and prepared for model development
- Presents unique challenges extending beyond gathering sufficient training examples

Data quality serves as the foundation that determines whether machine learning systems succeed or fail in production environments



Complete ML lifecycle through two parallel pipelines:

- The data pipeline (green, top row) transforms raw inputs through collection, ingestion, analysis, labeling, validation, and preparation into ML-ready datasets
- The model development pipeline (blue, bottom row) takes these datasets through training, evaluation, validation, and deployment to create production systems

The curved feedback arrows show how deployment insights trigger data refinements, creating continuous improvement cycles that distinguish ML from traditional linear development.

Table of Contents

- 1 Data-centric AI Development
- 2 Data Engineering
- 3 Data Quality & Data Reliability
- 4 Data Scalability
- 5 Data Governance
- 6 Scoping

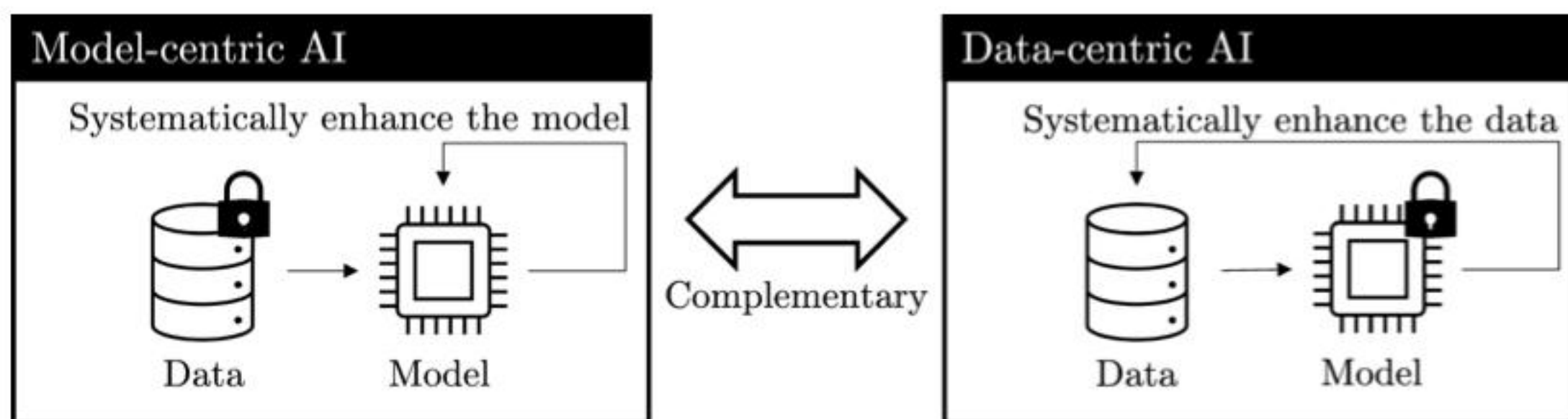
Model-centric vs. data-centric

Model-Centric AI

The paradigm focuses on the choice of a suitable model type, architecture, and hyperparameters from a wide range of possibilities for building effective and efficient AI-based systems.

Data-centric AI

The paradigm emphasizes the systematic design and engineering of data that are essential for building effective and efficient AI-based systems.



Model-centric vs. data-centric

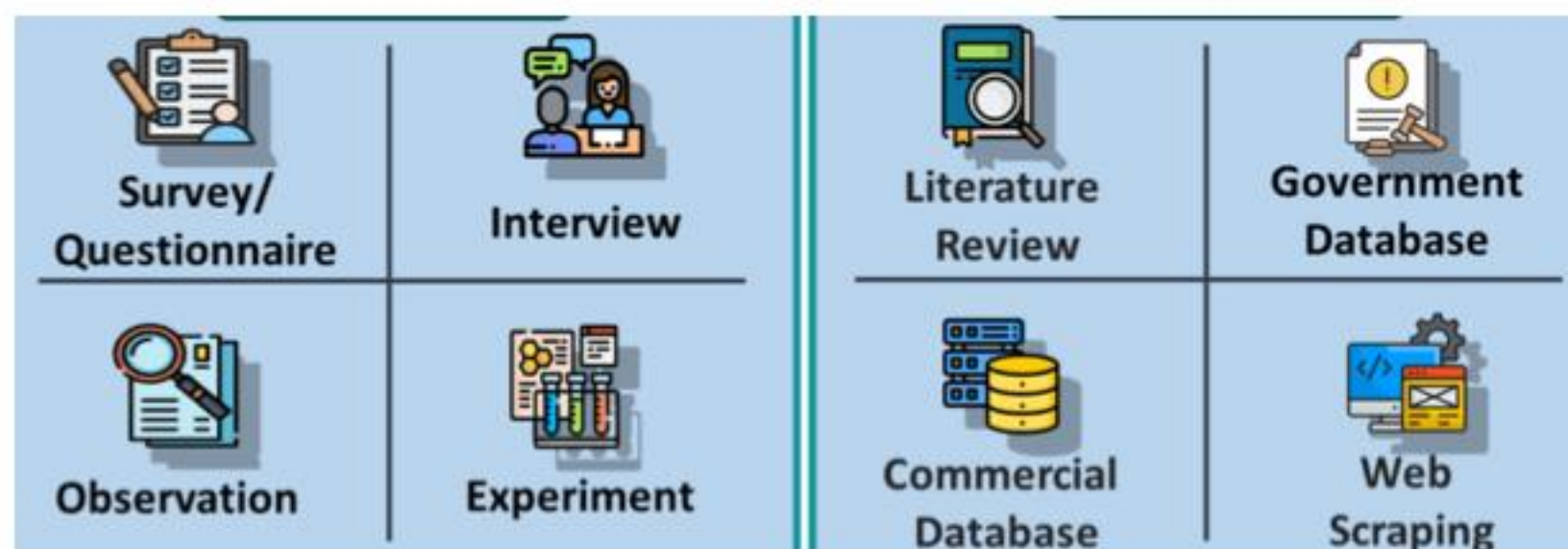
Aspect	Model-Centric AI	Data-Centric AI
Focus	Improving model architectures and algorithms	Improving data quality, consistency, and coverage
Approach	Iterate on model design, hyperparameters, and training procedures	Iterate on data collection, labeling, cleaning, and augmentation
Assumption	Data is fixed; performance gains from better models	Model is relatively fixed; performance gains from better data
Techniques	Deeper networks, ensembling, architecture search	Data cleaning, re-labeling, balancing, augmentation
Scalability	Often limited by model complexity and computational cost	Scales well by improving datasets without increasing model size
Robustness	May overfit noisy or biased data	Improves generalization and robustness through better data
Interpretability	Often reduced with complex models	Enhanced through cleaner and more consistent data

Table of Contents

- 1 Data-centric AI Development
- 2 Data Engineering**
- 3 Data Quality & Data Reliability
- 4 Data Scalability
- 5 Data Governance
- 6 Scoping

Data

- Refers to raw observations, measurements, or records collected from the real world that can be stored, processed, and analyzed to extract information or train models.
 - Structured data: tables, databases
 - Semi-structured data: JSON, logs
 - Unstructured: text, images, audio, video
- May be labeled or unlabeled. For labelled data:
 - Input
 - Target label
- Collected from sources such as sensors, users, systems, or experiments



Data Engineering

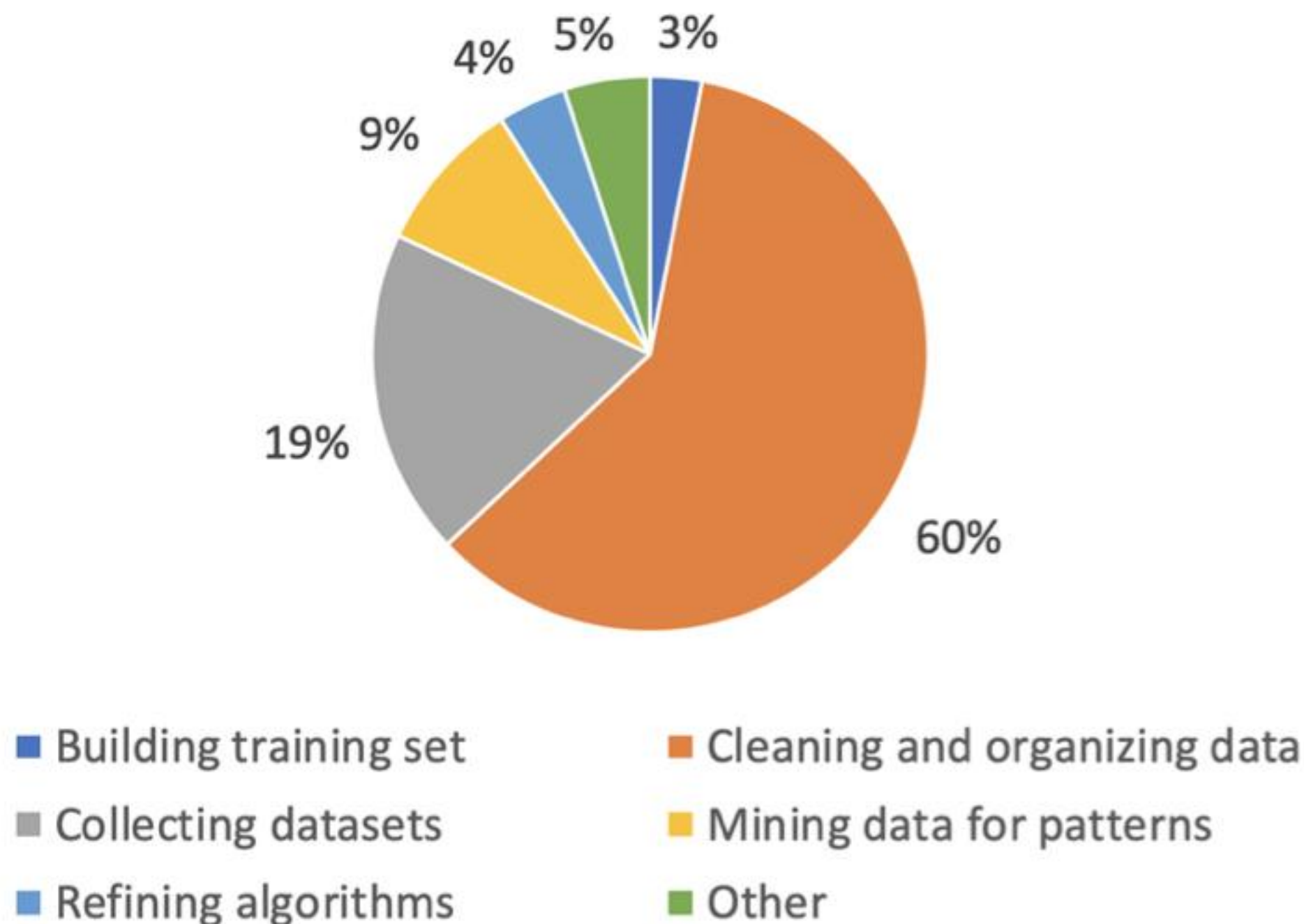
Data Engineering is the systematic discipline of designing and maintaining data infrastructure that transforms raw data into reliable, accessible, and analysis-ready datasets through principled acquisition, processing, storage, and governance practices.

No algorithm can overcome poor data, but excellent data engineering enables even simple models to achieve remarkable results.

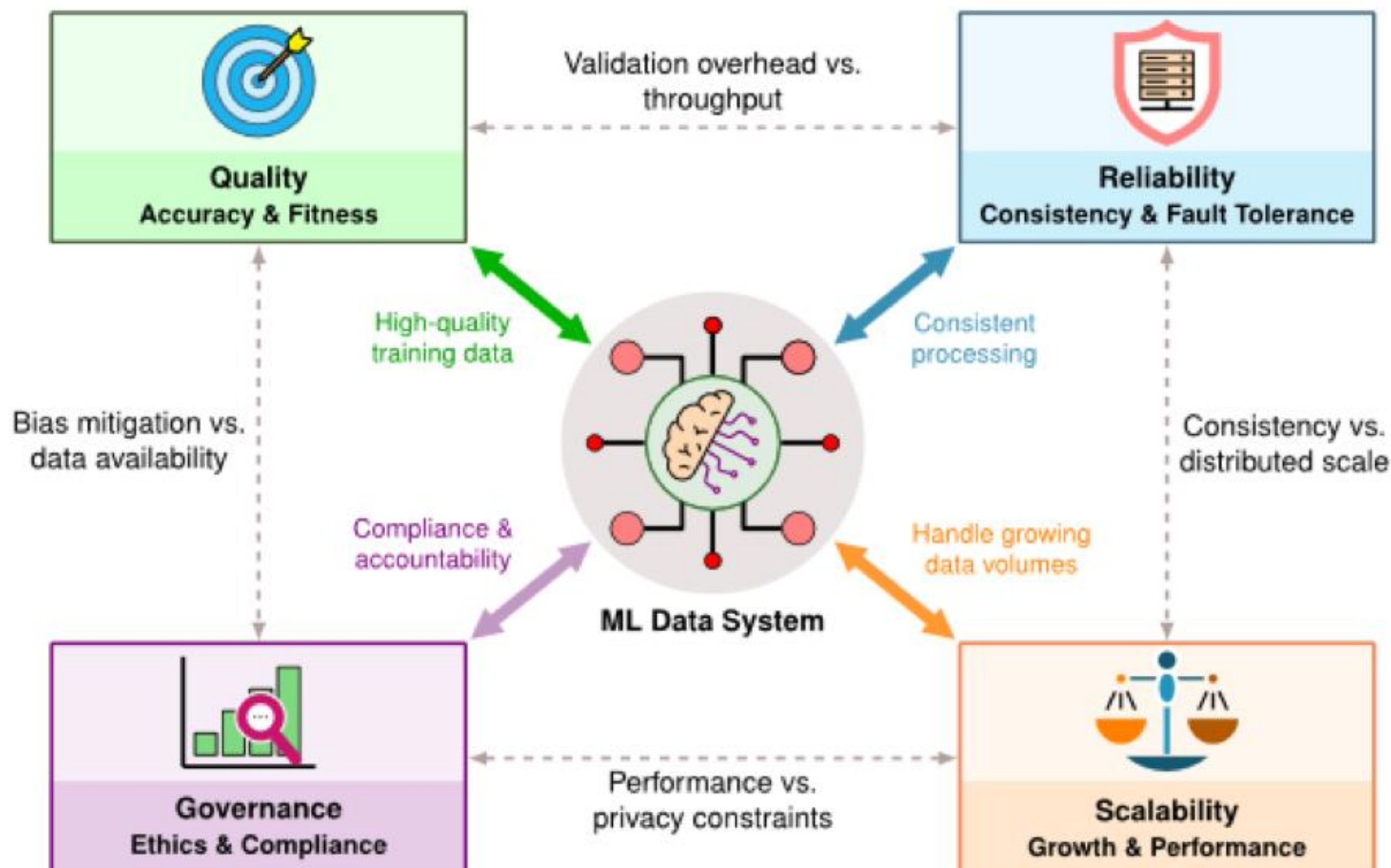
Aspect	Software Engineering	Data Engineering
Focus	Building and maintaining software applications	Designing, building, and maintaining data pipelines and infrastructure
Objective	Deliver reliable, scalable, and maintainable software systems	Ensure reliable, scalable, and high-quality data flow for analytics and ML
Artifacts	Source code, APIs, services, applications	Data pipelines, data warehouses, data lakes, ETL/ELT workflows
Inputs	User requirements, business logic	Raw, semi-structured, and structured data
Outputs	Software products and services	Cleaned, transformed, and accessible data
Skills	Programming, software architecture, testing, version control	Data modeling, ETL, distributed systems, data quality management
Scalability	Application performance and user load	Data volume, velocity, and variety
Quality Metrics	Code correctness, maintainability, latency, uptime	Data completeness, accuracy, freshness, and consistency

Data scientist time allocation

What data scientists spend the most time doing



Data Engineering Pillars Across Pipeline Stages



Data Engineering Pillars Across Pipeline Stages

Ensure data engineering systems operate within ethical, legal, and business constraints.

Stage	Quality	Reliability	Scalability	Governance
Acquisition	Representative sampling, bias detection	Diverse sources, redundant collection strategies	Web scraping, synthetic data generation	Consent, anonymization, ethical sourcing
Ingestion	Schema validation, data profiling	Dead letter queues, graceful degradation	Batch vs stream processing, autoscaling pipelines	Access controls, audit logs, data lineage
Processing	Consistency validation, training-serving parity	Idempotent transformations, retry mechanisms	Distributed frameworks, horizontal scaling	Lineage tracking, privacy preservation, bias monitoring
Storage	Data validation checks, freshness monitoring	Backups, replication, disaster recovery	Tiered storage, partitioning, compression optimization	Access audits, encryption, retention policies

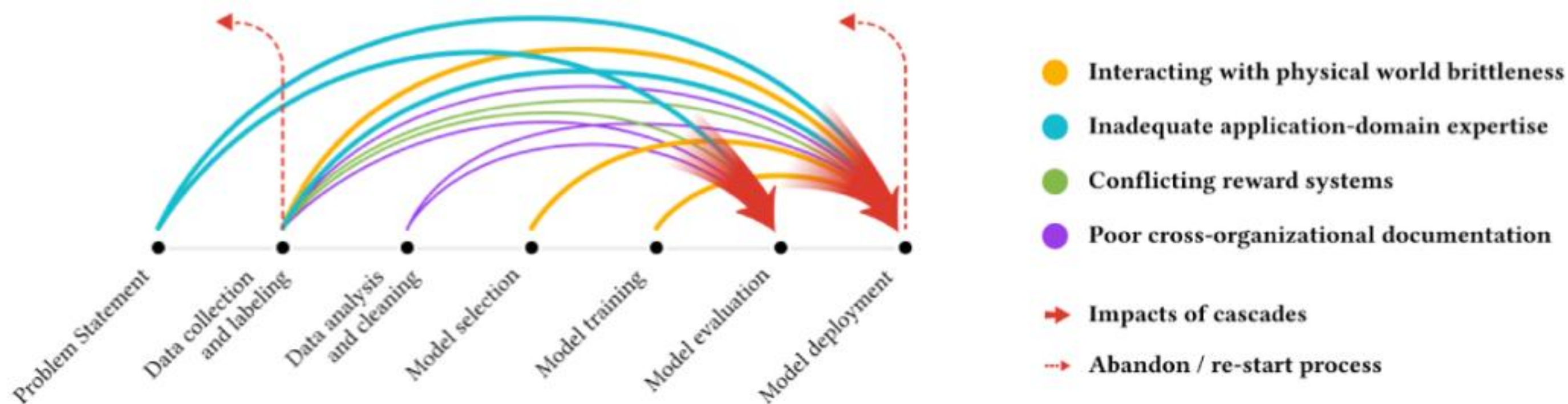
Table of Contents

- 1 Data-centric AI Development
- 2 Data Engineering
- 3 Data Quality & Data Reliability**
- 4 Data Scalability
- 5 Data Governance
- 6 Scoping

Data Cascades

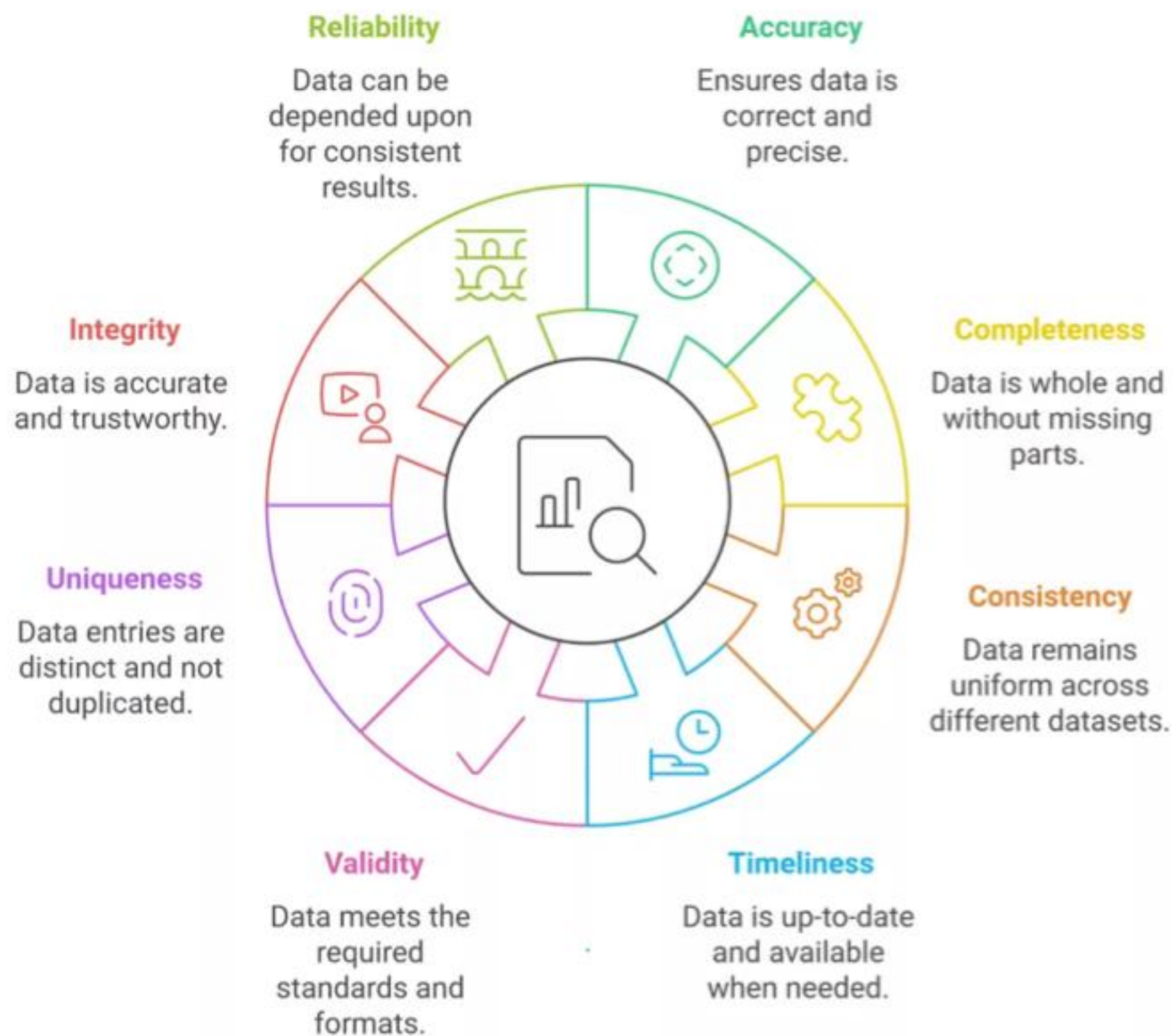
Data Cascades

Data Cascades is the phenomenon where poor data quality in early stages amplifies throughout the entire pipeline, causing downstream model failures, project termination, and potential user harm



Recognizing these cascades motivates proactive investment in data engineering and quality control to mitigate risks and ensure reliable system performance.

Data Quality



- Data reliability refers to the trustworthiness and consistency of data used in ML systems
- Reliable data produces stable, accurate, and reproducible model behavior

Unreliable data leads to:

- Poor model performance, even well-designed models
- Unexpected behavior in production
- Loss of trust in ML systems

Good models cannot compensate for bad data.

Common Data Quality/Reliability Issues

- Missing data (incomplete records)
- Incorrect data (sensor errors, wrong labels)
- Inconsistent data (schema or format changes)
- Duplicate data
- Outdated data

These issues can silently degrade model performance.

Some causes:

- Data integration issues
- Human errors
- System and software limitations
- Poor data governance

Label Inconsistency



Labeling instructions: "Use bounding boxes to indicate the position of iguanas"

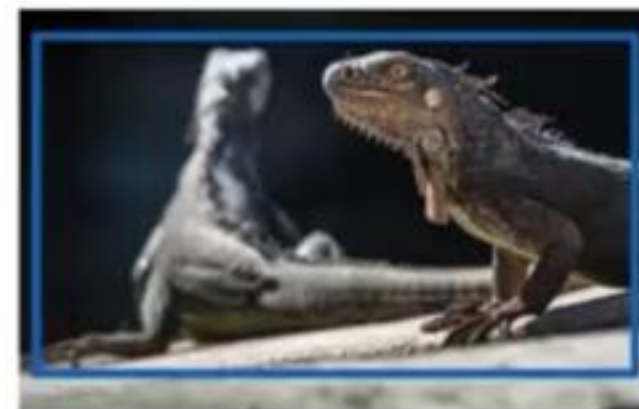


Figure: Inconsistent labeling. Source: deeplearning.ai

Data Reliability vs. Model Reliability

Data Reliability

- *What:* Consistency and stability of data across time and measurements
- *Focus:* Data pipelines, completeness, accuracy, freshness
- *Example:* Sales totals remain unchanged when no new sales occur

Model Reliability (Robustness)

- *What:* Consistent and trustworthy predictions on unseen data
- *Focus:* Performance metrics, robustness to drift, validation
- *Example:* Fraud model performs reliably across users and days

Key Differences & Interdependence

- Data reliability concerns **inputs**; model reliability concerns **outputs**
- Unreliable data causes inconsistent results; weak models fail even with good data

Quiz: Is This a Data or Model Issue?

For each scenario, decide whether the problem is mainly caused by **Data** or the **Model**.

- 1 A face recognition model suddenly performs poorly after a camera upgrade.
- 2 A spam classifier predicts “*not spam*” for almost all emails.
- 3 A credit risk model trained last year underestimates default rates this year.
- 4 A model performs well on training data but poorly on test data.
- 5 An image classifier fails when images are rotated or poorly lit.

Discuss your answers with a neighbor and justify your reasoning.

Table of Contents

- 1 Data-centric AI Development
- 2 Data Engineering
- 3 Data Quality & Data Reliability
- 4 Data Scalability**
- 5 Data Governance
- 6 Scoping

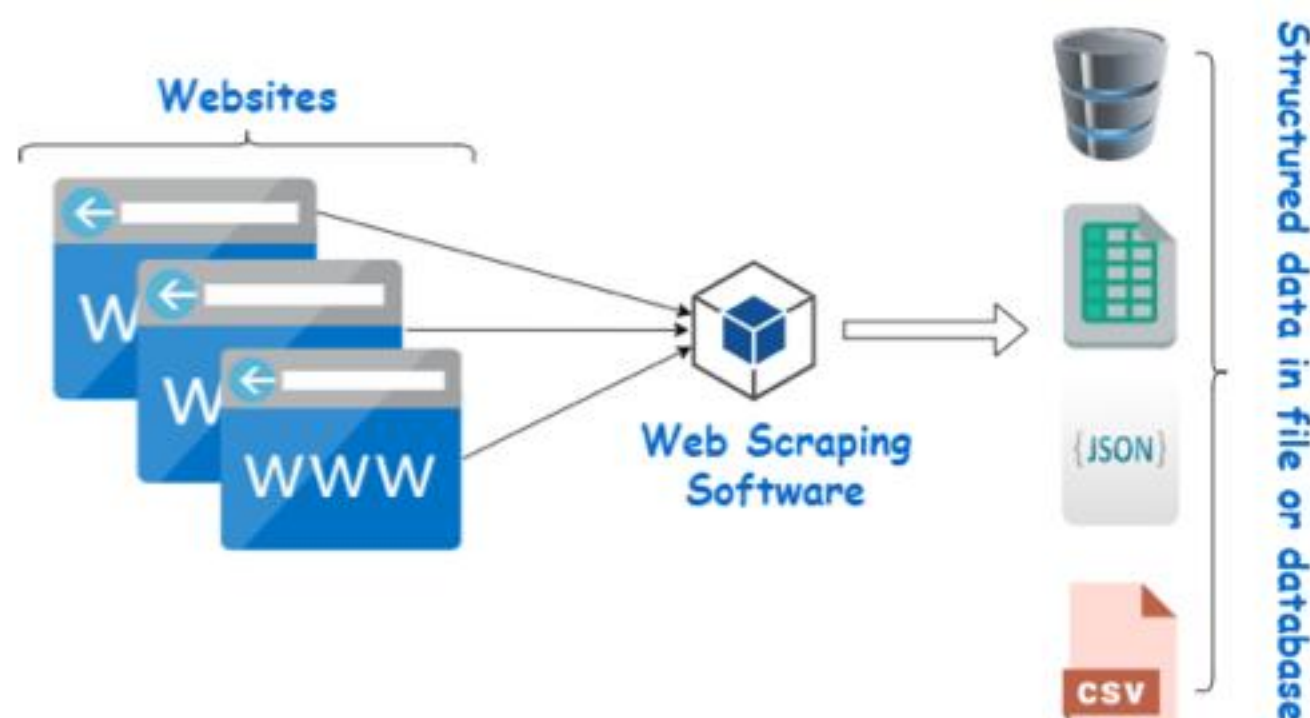
Types of Data Problems

	Structured Data	Unstructured Data
Small Data	• Student grades prediction • Credit approval with 100 training examples • Sales forecasting for a small shop	• Medical image diagnosis • Small image dataset for fruit quality inspection • Email spam detection with 200 examples
Big Data	• Sale forecasting with e-commerce sales data • Demand forecasting with airline reservations • Stock price prediction with financial market data	• Image search on billions of photos • Speech recognition from audio streams • Social media text analysis

- As data becomes larger and less structured, both modeling and engineering complexity increase
- Creating accurate, well-curated datasets faces inherent scaling limitations
- Cost-effective acquisition strategy at thousand-example scale often becomes prohibitive at million-example scale
- Production ML systems often require continuous data collection to maintain relevance and performance

Web scraping

- Automatically load, crawl and extract data from multiple pages of websites based on requirement
- Offer a powerful approach to gathering training data at scale



Constraints:

- Ensure compliance with terms of service and copyright laws (legal and ethical constraints)
- Rate limiting by websites slow data collection, inconsistent data (technical constraints)

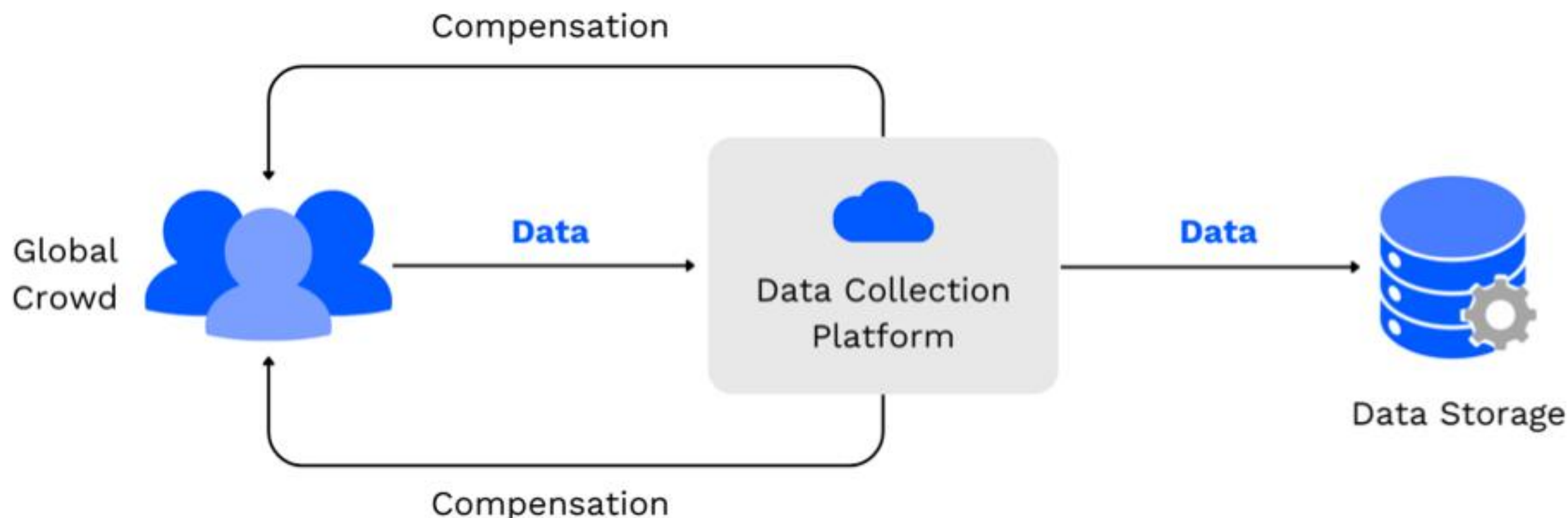
Crowdsourcing

Distribute microtasks to a large audience. For example

- ImageNet was built by distributing image labeling tasks to contributors via Amazon Mechanical Turk
- Waze uses crowdsourced data from its users to provide real-time traffic updates, route suggestions, and incident reporting

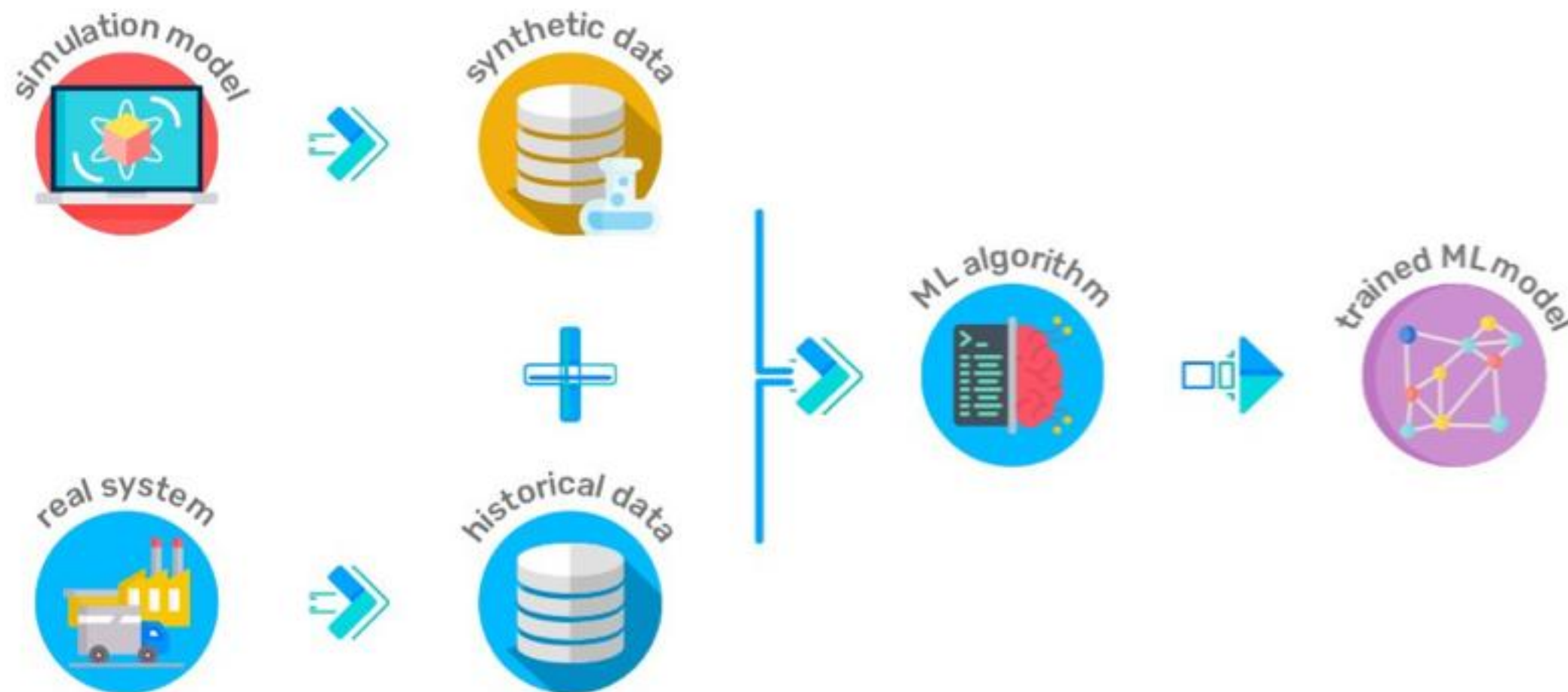
Flexibility: tasks can be adjusted dynamically based on initial results

- Google's reCAPTCHA system uses crowdsourcing to verify human users while simultaneously labeling datasets for training machine learning models



Synthetic Data Generation

- Create unlimited training examples through algorithmic generation rather than manual collection, removing human labor from data acquisition
- Modern AI systems can produce data that closely resembles real-world distributions, making it suitable for applications ranging from computer vision to natural language processing
- Particularly valuable in domains where obtaining real-world data is either impractical or costly, such as the automotive industry



Example

Brainstorm a list of data sources for speech recognition

Source	Amount	Cost	Time
Owned	100h	\$0	0
Crowdsourced – Reading	1000h	\$10,000	14 days
Pay for labels	100h	\$6,000	7 days
Purchase data	1000h	\$10,000	1 day

Table: Comparison of data sourcing strategies by cost and time

Other factors: Data quality, privacy, regulatory constraints

Who is qualified to label?

- ① Domain experts
- ② Reasonable annotators
- ③ Almost no one

In the following scenarios:

- Speech recognition
- Medical image diagnosis
- Recommendation system
- Legal judgments

Batch Ingestion

Collect data in groups over a specified period before processing

- Real-time data processing is not critical
- Data can be processed at scheduled intervals
- Enable efficient use of computational resources by amortizing startup costs across large data volumes

Example: A retail company might use batch ingestion to process daily sales data overnight, updating ML models for inventory prediction each morning

Stream Ingestion

Process data in real-time as it arrives

- Require immediate data processing
- Data loses value quickly, and systems need to respond to events as they occur

Example: A financial institution might use stream ingestion for real-time fraud detection, processing each transaction as it occurs to flag suspicious activity immediately

- Must handle backpressure when downstream systems cannot keep pace—when a sudden traffic spike produces data faster than processing capacity
- Maximum acceptable delays between data generation and availability for processing were specified by Data freshness Service Level Agreements (SLAs).

Batch vs. Stream Data Ingestion

Aspect	Batch Ingestion	Stream Ingestion
Basic Idea	Data is collected and processed in groups	Data is processed continuously as it arrives
Timing	Periodic (hourly, daily, weekly)	Real-time or near real-time
Data Volume	Large chunks of data at once	Small events, one by one
Latency	High (minutes to hours)	Low (milliseconds to seconds)
Typical Use Cases	Reports, analytics, model training	Fraud detection, monitoring, alerts
Complexity	Easier to build and maintain	More complex to design and operate
Common Tools	SQL, Hadoop, Spark (batch)	Kafka, Spark Streaming, Flink
Failure Handling	Easy to retry whole batches	Requires careful handling of lost or delayed data
Cost	Usually cheaper to run	Often more expensive due to always-on systems

Distributed Data Processing

Split large datasets and computations across multiple machines (nodes) to improve speed, scalability, and fault tolerance.

- When single machines cannot handle data volumes or processing complexity
- Instead of one computer processing 1 TB of data in 10 hours, 10 computers process 100 GB each in about 1 hour

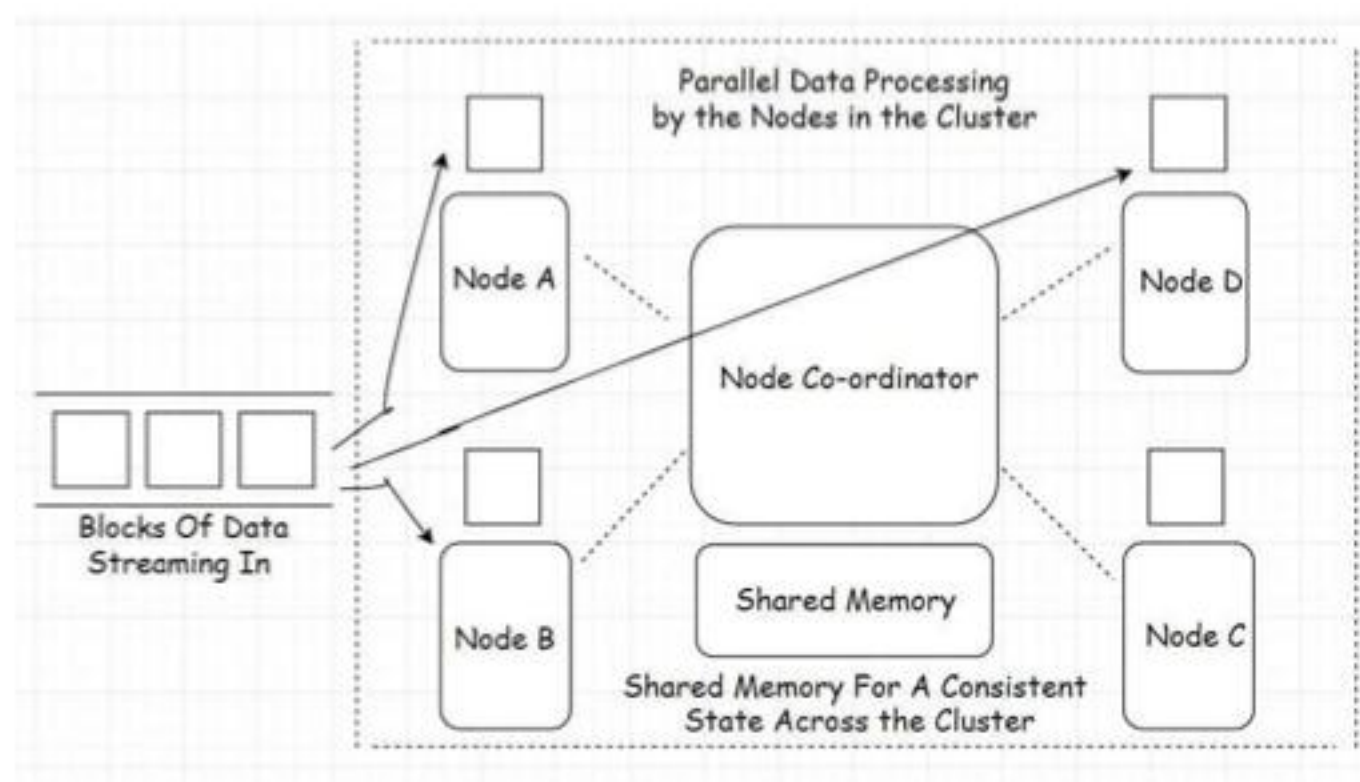


Figure: Each node processes a portion of the data in parallel. Coordinator / Master assigns tasks and collects results

Distributed Data Processing

Challenges

- Data must be partitioned across multiple computing resources, introducing coordination overhead
- Is limited by network round-trip times: local operations complete in microseconds while network coordination requires milliseconds

Common Technologies

- Batch: Hadoop, Spark
- Streaming: Kafka, Spark Streaming, Flink
- Cloud: AWS EMR, Google Dataflow, Azure Databricks

Table of Contents

- 1 Data-centric AI Development
- 2 Data Engineering
- 3 Data Quality & Data Reliability
- 4 Data Scalability
- 5 Data Governance**
- 6 Scoping

Data Governance



- Blue: organizational responsibilities, policies, documentation, and tools that enable people in the organization to understand data assets and policies and then leverage data to strategic advantage
- Green: hybrid business, data, and technical practices that drive sourcing and managing data that can be used for analytics, APIs, applications, and other business opportunities

Data Governance Principles

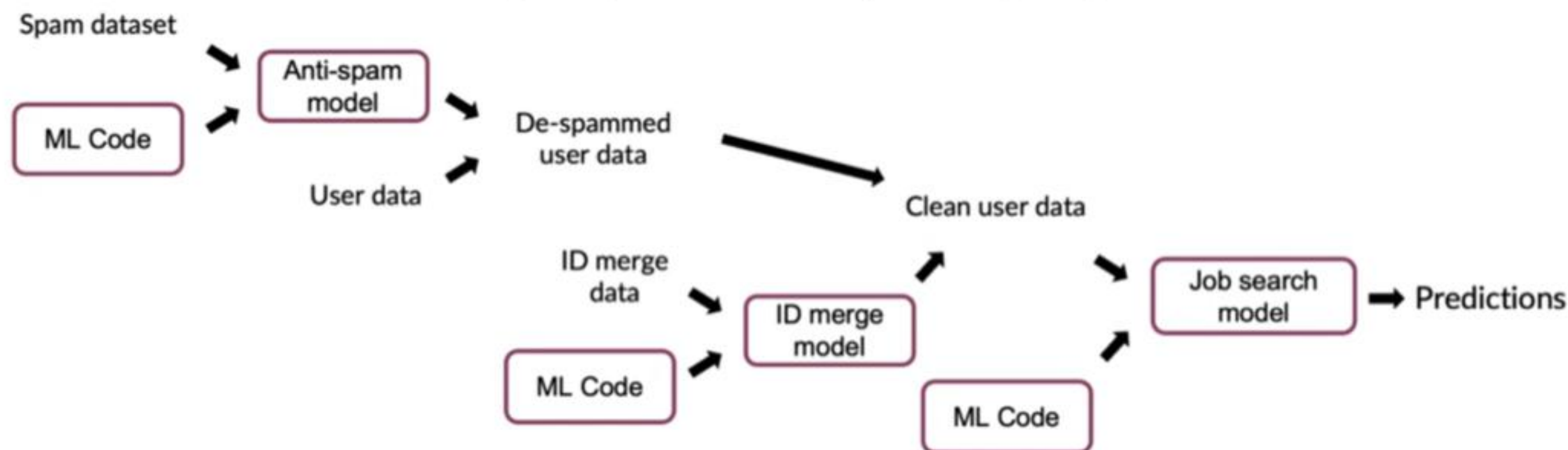
Understand what data flows through the system, how it transforms, and who accesses it

- Ensure data engineering systems operate within ethical, legal, and business constraints
- Maintain transparency and accountability
- Protect user privacy and maintain security throughout ML lifecycle

Foundational requirements that shape every technical decision from the outset, not afterthoughts to be applied later

Data Pipeline Example

Task: Predict if someone is looking for a job. (x = user data, y = looking for a job?)



It is essential to track data provenance and lineage to ensure the system is maintainable, especially when upstream data must be changed.

Data lineage

Data lineage tracking captures the complete provenance (essential metadata) of every dataset.

- Which raw sources contributed data (data provenance)
- What transformations were applied when processing occurred
- What version of processing code executed

Data lineage is essential for debugging model behavior and ensuring reproducibility.



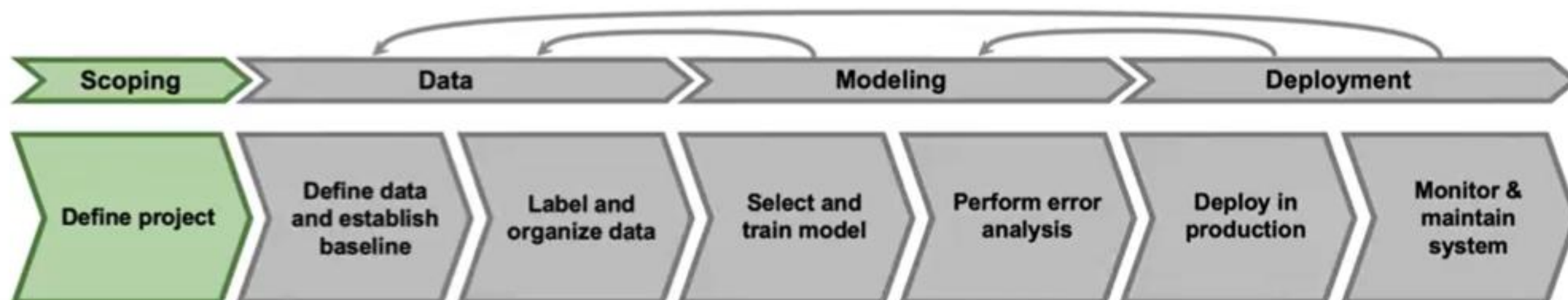
Audit trails

- Complement lineage by recording who accessed data and when.
- Compliance with regulations such as GDPR and CCPA
- Protect the organization's reputation, avoid potential financial and legal consequences, and increase stakeholder trust

Table of Contents

- 1 Data-centric AI Development
- 2 Data Engineering
- 3 Data Quality & Data Reliability
- 4 Data Scalability
- 5 Data Governance
- 6 Scoping**

Problem Definition



Scoping example:  Ecommerce retailer looking to increase sales

- Better recommender system
- Better search
- Improve catalog data
- Inventory management
- Price optimization

Questions:

- What project should we work on?
- What are the metrics for success?
- What are the resources (data, time, people) needed?

Problem Identification vs. Solution

Problem (What to Achieve)	Solution (How to Achieve)
Increase conversion	Search, recommendations
Reduce inventory	Demand prediction, marketing
Increase margin (profit per item)	Optimizing what to sell (e.g., merchandising), recommend bundles

Feasibility

Answer the question: Is this project technically possible?

- Use external benchmark (literature, competitor, other company)

	Structured Data	Unstructured Data
New	Availability of predictive features	HLP
Existing	New predictive features, History of project	HLP, History of project



Source: deeplearning.ai

Decide:

- ① Highly Predictive
- ② Weakly Predictive
- ③ Not Predictive / Leaky

for each of the following scenarios

- Predicting **house prices**:
 - House size (m^2)
 - Wall color
 - ZIP code
- Predicting **student exam performance**:
 - Hours studied
 - Student ID number
 - Attendance rate

- Predicting **credit default**:
 - Past repayment history
 - Browser type
 - Credit utilization rate
- Predicting **fraudulent transactions**:
 - Transaction amount
 - Time of day
 - Label added after investigation

Which features might cause **data leakage**?

Data leakage refers to the unintentional exposure of sensitive information, either

- by accidental release to unauthorized parties, or
- by including future/target information in a machine learning model's training data, causing it to perform well in testing but poorly in reality

Discussion

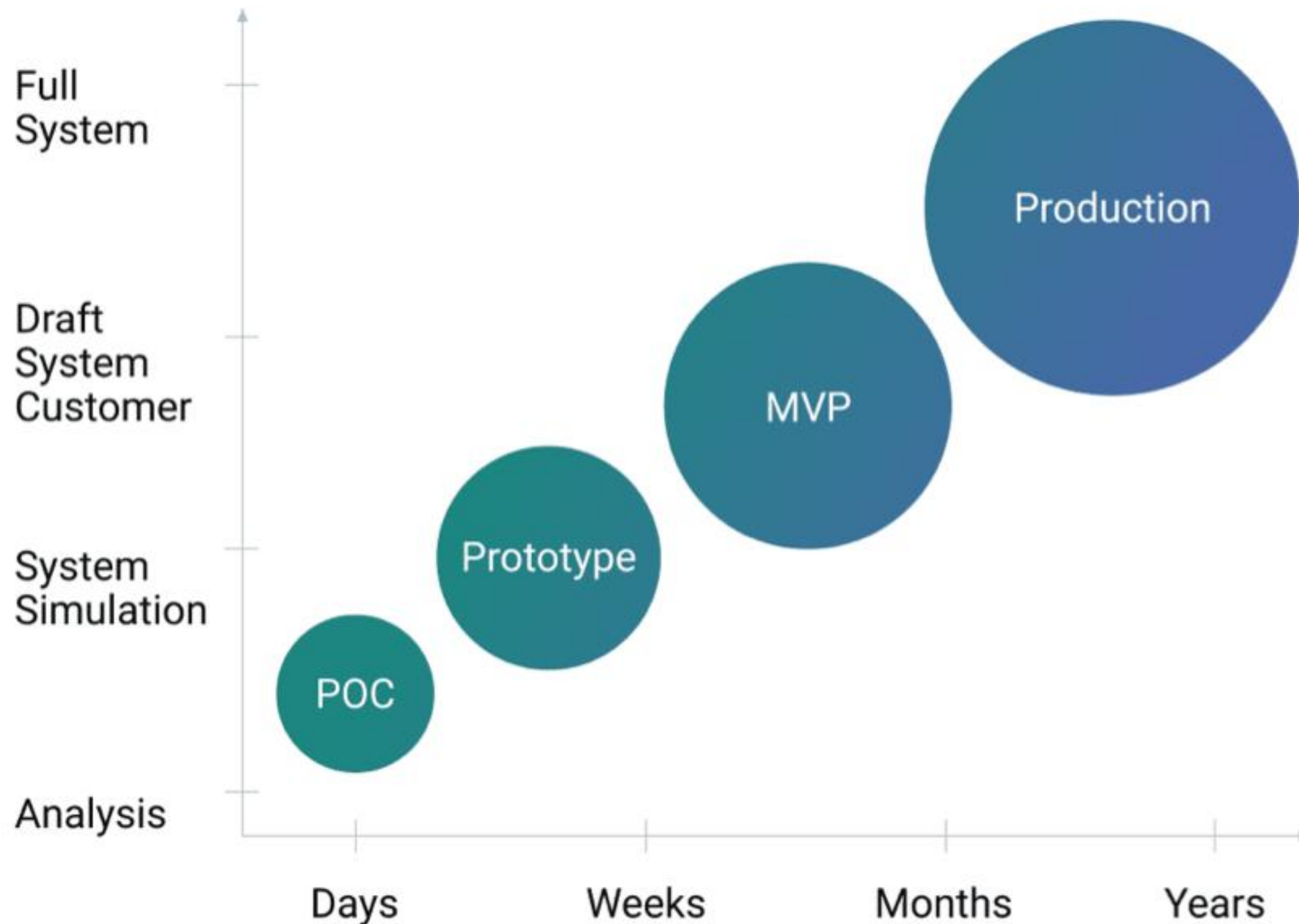
Decide whether one has predictive features.

- Given past purchases, predict future purchases ✓
- Given weather data, predict shopping mall foot traffic ✓
- Given DNA information, predict heart disease ?
- Given social media chatter, predict demand for a clothing style ?
- Given history of a stock's price, predict its future price ✗

Discussion

- Do we have sufficient signal in the data?
- Is the problem stable over time?
- Are there ethical or causal limitations?

Development Stages



PoC (Proof-of-Concept)

Early in the discovery phase, before significant investment.

- Goal: Validate whether a specific application is possible and worth building
- Focus: Technical feasibility, core mechanics, overcoming specific challenges, choosing tech stacks
- Output: Often a simple, non-visual demonstration or code snippet; internal-facing

PoC is only about 20% of the total journey. The real engineering effort begins thereafter.

Prototype

Refine design and test user interaction after the feasibility of PoC is confirmed

- Goal: Show how the final product will look, feel, and function for users
- Focus: User interface (UI), user flow, design, and gathering early user feedback
- Output: An interactive, early-stage model (low-fidelity wireframe to high-fidelity clickable mockup)

PoC vs. Prototype

Feature	Proof of Concept (PoC)	Prototype
Primary Question	Can it be built? (Feasibility)	How does it work or look? (Usability)
Purpose	Verify technical viability	Visualize functionality, test user experience, and gather feedback
Audience	Internal team, technical stakeholders	Users, investors, and broader stakeholders
Complexity	Low; focuses on core functionality	Medium; demonstrates design and interaction flow
Development Stage	Early discovery (pre-prototype)	Early or seeding phase (post-PoC)

Minimum Viable Product (MVP)

Simplest version of a product with core features, designed to test core assumptions and gather rapid user feedback

- Goal: Validate core hypotheses about the product idea, assess market demand, and learn from real users quickly
- Focus: Core features that solve the primary user problem, enabling maximum learning with minimum effort
- Outcome: Data and validated learning to guide future development, preventing wasted resources on unwanted features

Production

- Goal: Deliver a complete, high-quality, and scalable solution that meets broad market needs and expectations
- Focus: Full feature set, robust performance, excellent user experience, security, and scalability for mass adoption
- Outcome: A commercially viable product that generates revenue and serves a large customer base

MVP vs. Production

Feature	Minimum Viable Product (MVP)	Production System
Purpose	Learn and validate assumptions	Deliver value and scale to users
Scope	Minimal set of core features	Full and complete feature set
Target Users	Early adopters and testers	Broad market and end users
Quality	Functional, potentially basic	Polished, robust, and reliable
Development Effort	Low initial investment	High development and maintenance cost
Risk Level	Low; focused on testing hypotheses	Higher; associated with market launch and scaling

Experiment

Explore the following notebook

`https://github.com/ageron/handson-ml3/blob/main/02\_end\_to\_end\_machine\_learning\_project.ipynb`

from the 2nd chapter in the book "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition" by Aurelien Geron.

Log the best model and then load it to predict the price of a given house. Print the parameters of the model.

References

- Sambasivan, Nithya, et al. *"Everyone wants to do the model work, not the data work": Data Cascades in High-Stakes AI*. proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. 2021.
- Treveil, Mark, et al. *Introducing MLOps*. O'Reilly Media, 2020. Link <https://www.oreilly.com/library/view/introducing-mlops/9781492083283/>
- Gift, Noah, and Alfredo Deza. *Practical MLOps*. O'Reilly Media, Inc., 2021. Link <https://learning.oreilly.com/library/view/practical-mlops/9781098103002/>